



Introduction au langage de programmation Python

Séance 2 - Créer ses propres outils : fonctions et modules



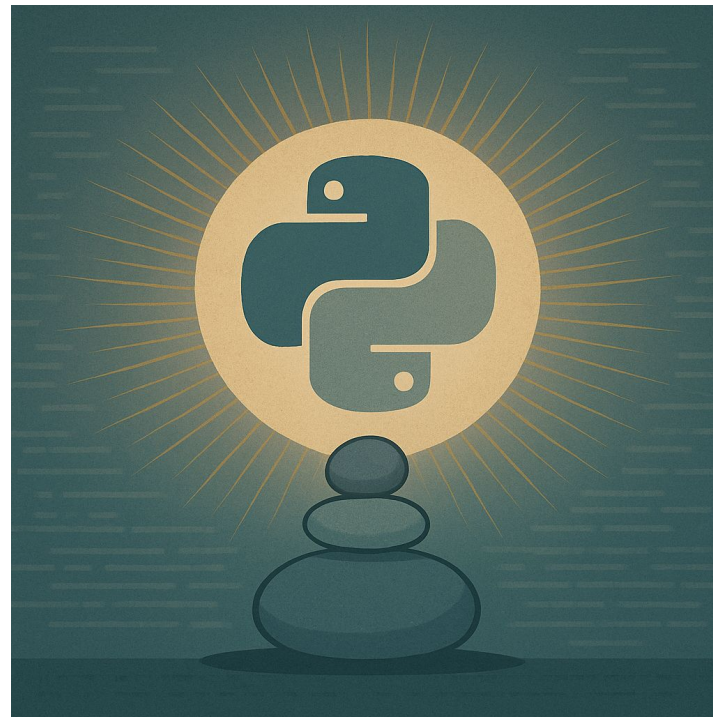
Programme du cours

Séance	Date	Sujet
1	29 septembre 2026	Les bases de la programmation
2	6 octobre 2026	Créer ses propres outils : fonctions et modules
3	13 octobre 2026	Programmation orientée objet 1 : créer ses propres objets
4	20 octobre 2026	Programmation orientée objet 2 : héritage et décorateurs
5	27 octobre 2026	Lire et enregistrer des données : JSON
6	3 novembre 2026	Du tableur à Python : CSV
7	10 novembre 2026	Explorer les collections numérisées : IIIF



Plan de la séance 2

- 1) Correction de l'interrogation
- 2) Les fonctions en Python
 - a) Définition : fonctions et méthodes
 - b) Structure d'une fonction
 - c) Renvoyer ou afficher : `return` et `print()`
 - d) Paramètres, arguments et valeurs par défaut
 - e) Nombre variable d'arguments : `*args` et `**kwargs`
 - f) Portée des variables
 - g) Fonction anonyme (`lambda`)
- 3) Documenter son code
 - a) Pourquoi commenter ?
 - b) Les commentaires avec `#`
 - c) Les docstrings
- 4) Les modules en Python
 - a) Qu'est-ce qu'un module ?
 - b) Importation de modules et conventions
 - c) Quelques modules utiles
 - d) Créer son propre module
 - e) Organiser ses modules : les packages





Correction de l'interrogation (1/2)

Q.	Réponse	Pourquoi
1	<code>int</code>	<code>1200</code> est un nombre entier. <code>1200.0</code> serait un <code>float</code> , <code>"1200"</code> un <code>str</code> .
2	<code>"dragon"</code>	Un index négatif compte depuis la fin : <code>-1</code> désigne le dernier élément.
3	<code>"III"</code>	Dans une tranche, la fin est exclue : <code>[0:3]</code> prend les index 0, 1 et 2.
4	Python renvoie une erreur	Un tuple est immuable : <code>TypeError: 'tuple' object does not support item assignment</code> .
5	<code>manuscrit["langue"] = "latin"</code>	On ajoute une clé en lui assignant une valeur. <code>append()</code> sert aux listes, <code>add()</code> aux sets.



Correction de l'interrogation (2/2)

Q.	Réponse	Pourquoi
6	3	Un set supprime les doublons : il reste {"lion", "dragon", "licorne"}.
7	3	<code>split(" ")</code> coupe aux deux espaces : ['De', 'la', 'colombe.']. Piège : 14 est la longueur de la chaîne, pas le nombre de mots.
8	XIIIe siecle	<code>1150 <= 1300</code> est vrai : la première condition vraie l'emporte, et le <code>elif</code> n'est jamais testé.
9	True	Avec <code>or</code> , une seule condition vraie suffit : <code>1200 < 1200</code> est faux, mais <code>numerise</code> vaut <code>True</code> .
10	dragon seulement	<code>break</code> arrête la boucle dès « licorne », avant le <code>print()</code> .



Les fonctions en Python

Définition

Une **fonction** est un bloc de code qui porte un nom et réalise une tâche précise. On l'écrit une seule fois, puis on l'**appelle** par son nom autant de fois qu'on veut, sans réécrire le code.

Pour reprendre l'image de la séance 1, les fonctions sont les **verbes** de Python. Jusqu'ici, on utilisait ceux du langage : `print()`, `len()`, `append()`. Aujourd'hui, on va créer les nôtres.

Structure type d'une fonction

Une fonction reçoit des **paramètres** (les entrées) et peut **renvoyer** une valeur (la sortie).

```
def nom_de_la_fonction(parametre1, parametre2):  
    """Décrire ce que fait la fonction."""  
    # instructions  
    print(resultat) # et/ou  
    return resultat
```

Fonction ou méthode ?

Les deux s'utilisent avec des parenthèses, mais elles ne s'écrivent pas de la même façon.

Une **fonction** s'applique à une donnée, placée entre les parenthèses :

```
len(titre)      # 23  
len(creatures)  # 4
```

Une **méthode** appartient à un type de donnée. Elle s'écrit :

```
titre.split()      # ['III', '14.', 'De', 'la', 'colombe']  
creatures.append("phénix") # ajoute "phénix" à la liste
```

Chaque type a ses propres méthodes. Une chaîne n'a pas de méthode `append()` :

```
titre.append("!")  
# AttributeError: 'str' object has no attribute 'append'
```



Structure d'une fonction

Exemple : le code du siècle devient une fonction

```
def siecle(date):  
    """Renvoyer le siècle d'une date sous forme de texte."""  
    if date <= 1100:  
        resultat = "XIe siècle ou avant"  
    elif date <= 1200:  
        resultat = "XIIe siècle"  
    elif date <= 1300:  
        resultat = "XIIIe siècle"  
    else:  
        resultat = "Après 1300"  
    return resultat
```

Appel de la fonction

```
siecle_aberdeen = siecle(manuscrit["date"])  
print(siecle_aberdeen) # XIIe siècle  
print(siecle(1210))    # XIIIe siècle
```

Déclaration

- Le mot-clé **def** est suivi du **nom de la fonction** et de ses **paramètres** entre parenthèses, ici date.
- Les deux-points : ouvrent le bloc de la fonction.

Corps de la fonction

- Les instructions sont **indentées** de 4 espaces.
- La **docstring**, entre triples guillemets `"""..."""`, décrit ce que fait la fonction.
- Le traitement reprend le **if/elif/else** de la séance 1.

Renvoi de la valeur

- **return** renvoie le résultat à l'endroit où la fonction a été appelée.

Appel de la fonction

- On appelle la fonction par son nom, en lui donnant un **argument** : `manuscrit["date"]`, puis 1210.
- Le résultat peut être rangé dans une variable (`siecle_aberdeen`) ou affiché directement.



Renvoyer ou afficher : `return` et `print()`

Deux instructions qui n'ont pas le même rôle

- `print()` **affiche** une valeur à l'écran, pour l'humain qui lit. La valeur n'est pas conservée.
- `return` **renvoie** une valeur au programme, à l'endroit où la fonction a été appelée. On peut alors la ranger dans une variable, la réutiliser, la combiner.

Une fonction sans `return` renvoie quand même quelque chose : la valeur spéciale `None`, qui signifie « rien ».

La même fonction, écrite avec `print()`

```
def siecle_affiche(date):  
    """Afficher le siècle d'une date."""  
    if date <= 1100:  
        print("XIe siècle ou avant")  
    elif date <= 1200:  
        print("XIIe siècle")  
    else:  
        print("Après 1300")
```

Avec `print()` : le résultat est perdu

```
valeur = siecle_affiche(1200)  
# Affiche : XIIe siècle  
print(valeur)  
# Affiche : None  
  
phrase = f"Le manuscrit date du {siecle_affiche(1200)}."  
print(phrase)  
# Affiche : XIIe siècle  
# puis : Le manuscrit date du None.
```

Avec `return` : le résultat est réutilisable

```
phrase = f"Le manuscrit date du {siecle(1200)}."  
print(phrase)  
# Affiche : Le manuscrit date du XIIe siècle.
```

À **retenir** : une fonction calcule et renvoie avec `return`. C'est à celui qui l'appelle de décider s'il veut afficher le résultat.



Paramètres, arguments et valeurs par défaut

Paramètres et arguments

- Les **paramètres** sont les noms déclarés entre les parenthèses de def. Ce sont des emplacements vides (ici : `cote`, `bibliotheque`, `date`, `langue`).
- Les **arguments** sont les valeurs réelles transmises au moment de l'appel (ici : `"MS 24"`, `1200...`).

Valeur par défaut

Un paramètre peut recevoir une valeur par défaut, utilisée si l'appel ne fournit pas d'argument pour lui.

```
def creer_notice(cote, bibliotheque, date, langue="latin"):
    """Renvoyer la notice d'un manuscrit sous forme de
    dictionnaire."""
    return {"cote": cote, "bibliotheque": bibliotheque,
            "date": date, "langue": langue}
```

Règle : les paramètres avec valeur par défaut se placent **après** les autres.

```
def creer_notice(langue="latin", cote): ...
# SyntaxError: parameter without a default follows parameter
with a default
```



Appeler une fonction : arguments positionnels et nommés

Arguments positionnels : ils sont associés aux paramètres dans l'ordre.

```
creer_notice("MS 24", "Aberdeen, University Library", 1200)  
# {'cote': 'MS 24', 'bibliotheque': 'Aberdeen, University Library',  
#  'date': 1200, 'langue': 'latin'}
```

Arguments nommés : on précise le nom du paramètre, et l'ordre n'a plus d'importance.

```
creer_notice(date=1210, cote="MS. Ashmole 1511",  
             bibliotheque="Oxford, Bodleian Library")
```

Remplacer la valeur par défaut

```
creer_notice("MS 24", "Aberdeen, University Library", 1200,  
             langue="anglo-normand")
```

Deux erreurs fréquentes

```
creer_notice("MS 24", "Aberdeen, University Library")  
# TypeError: creer_notice() missing 1 required positional  
argument: 'date'
```

```
creer_notice(cote="MS 24", "Aberdeen, University Library",  
             1200)  
# SyntaxError: positional argument follows keyword argument
```



Quand créer un paramètre ?

La règle

Un paramètre sert à transmettre une information qui **change d'un appel à l'autre**. La fonction ne peut pas la décider seule.

Le test

Peut-on imaginer un deuxième appel, dans notre projet, avec une autre valeur ?

- **Oui** : c'est un paramètre. La date de `siecle()` vaut 1200 pour Aberdeen et 1210 pour Oxford.
- **Non** : ce n'est pas un paramètre. La valeur reste fixe, dans la fonction ou dans un module partagé.

Le piège : trop de paramètres

Un paramètre qui ne devrait jamais changer donne une fausse impression de souplesse. Il ouvre aussi une occasion de se tromper, et Python ne prévient pas avant l'exécution.

Trop de paramètres

```
def cote_complete(manuscrit, cle_cote="cote"):
    return f"{manuscrit['bibliotheque']}, {manuscrit[cle_cote]}"
```

```
cote_complete(manuscrit, cle_cote="Cote")
# KeyError: 'Cote'
```

La clé "cote" est fixée par nos conventions et ne change jamais. En faire un paramètre ne sert qu'à permettre l'erreur.

Juste ce qu'il faut

```
def cote_complete(manuscrit):
    """Renvoyer la bibliothèque et la cote d'un manuscrit."""
    return f"{manuscrit['bibliotheque']}, {manuscrit['cote']}"
```

```
cote_complete(manuscrit)
# 'Aberdeen, University Library, MS 24'
```



Nombre variable d'arguments : `*args` et `**kwargs`

`*args` : autant d'arguments positionnels qu'on veut

L'étoile `*` regroupe tous les arguments positionnels supplémentaires dans un **tuple**.

```
def decrire_creatures(cote, *noms):  
    """Renvoyer une phrase sur les créatures d'un manuscrit."""  
    return f"{cote} : {len(noms)} créatures {noms}"
```

```
decrire_creatures("MS 24", "dragon", "licorne", "phénix")  
# 'MS 24 : 3 créatures ('dragon', 'licorne', 'phénix)'
```

`noms` est un tuple : il est donc immuable (séance 1).

`**kwargs` : autant d'arguments nommés qu'on veut

La double étoile `**` regroupe tous les arguments nommés supplémentaires dans un **dictionnaire**.

```
def notice_libre(**infos):  
    """Renvoyer une notice construite à partir des arguments  
    nommés."""  
    return infos  
notice_libre(cote="MS 24", date=1200, numerise=True)  
# {'cote': 'MS 24', 'date': 1200, 'numerise': True}
```

C'est utile quand on ne sait pas à l'avance quelles informations on aura sur un manuscrit.

Ordre obligatoire : les paramètres classiques, puis `*args`, puis `**kwargs`.

```
def f(cote, *noms, **infos): ...
```

`args` et `kwargs` ne sont que des noms d'usage : seules les étoiles comptent. On les retrouvera en séance 4 avec les décorateurs.



Portée des variables

Variable locale

Une variable créée à l'intérieur d'une fonction n'existe que pendant l'exécution de cette fonction. Elle disparaît ensuite.

```
siecle(1200)    # 'XIIe siècle'
print(resultat)
# NameError: name 'resultat' is not defined
```

`resultat` est locale à `siecle()` : on ne peut pas l'utiliser en dehors.

Variable globale

Une variable créée **en dehors** de toute fonction (dans le notebook ou le script) est globale. Une fonction peut la lire.

```
def est_au_corpus(nom):
    return nom in creatures # lit la liste globale
```

```
est_au_corpus("licorne") # True
```

Une affectation dans une fonction crée une variable locale

```
titre = "III, 14. De la colombe."
def changer_titre():
    titre = "Titre modifié" # nouvelle variable locale
    return titre
```

```
changer_titre() # 'Titre modifié'
print(titre)    # III, 14. De la colombe.
```

La variable globale `titre` n'a pas changé : les deux `titre` sont des variables différentes, qui portent le même nom.

Bonne pratique

- Une fonction reçoit ce dont elle a besoin par ses **paramètres** et transmet son résultat par `return`.
- Il vaut mieux éviter qu'elle dépende de variables globales : `est_au_corpus(nom, creatures)` est plus clair et plus facile à réutiliser.



Pourquoi utiliser des fonctions ?

Les avantages

- **Réutilisabilité** : on écrit `siecle()` une fois, et elle sert pour 2 manuscrits comme pour 200.
- **Lisibilité** : `siecle(ms["date"])` se lit comme une phrase, alors qu'un bloc `if/elif/else` répété à chaque fois devient vite illisible.
- **Maintenance** : si l'on corrige un seuil dans `siecle()`, la correction s'applique partout où la fonction est appelée.
- **Organisation** : un programme se découpe en petites tâches nommées, chacune testable séparément.
- **Collaboration** : chacun peut utiliser une fonction écrite par un autre sans en lire le détail, grâce à son nom et à sa docstring.

Exemple

```
corpus = [manuscrit, manuscrit_oxford]

for ms in corpus:
    print(ms["cote"], ":", siecle(ms["date"]))
# MS 24 : XIIe siècle
# MS. Ashmole 1511 : XIIIe siècle
```

Bonnes pratiques

- **Une fonction, une tâche** : `siecle()` calcule le siècle, et rien d'autre.
- **Des noms explicites**, pour la fonction comme pour ses paramètres : on écrit `siecle(date)` ou `cote_complete(manuscrit)`, pas `f(x)`.
- **Des noms sans accents ni espaces**, comme pour nos variables et nos clés.
- **Des paramètres en entrée, un `return` en sortie**
- **Une docstring** pour dire ce que fait la fonction



Fonction anonyme (**lambda**)

Définition

Une fonction **lambda** est une fonction **anonyme**, sans nom, écrite en **une seule ligne**. On l'utilise pour une petite opération dont on n'a besoin qu'à un seul endroit.

Syntaxe

lambda parametres: expression

- **lambda** est le mot-clé qui crée la fonction.
- **parametres** désigne les entrées, de zéro à plusieurs.
- **expression** est le calcul effectué. Son résultat est renvoyé automatiquement, sans **return**.

Équivalence avec **def**

```
lambda ms: ms["date"]
```

```
def date_du_manuscrit(ms):  
    return ms["date"]
```

Usage principal : passer une fonction en argument

sorted() accepte un argument nommé **key** : c'est une fonction appliquée à chaque élément pour décider de l'ordre.

```
sorted(creatures, key=lambda nom: len(nom))  
# ['lion', 'dragon', 'phénix', 'licorne', 'panthère']
```

Ici, **key=len** suffirait : **len** est déjà une fonction, et on peut la passer telle quelle.

Trier le corpus du plus récent au plus ancien

```
corpus_tries = sorted(corpus,  
                      key=lambda ms: ms["date"],  
                      reverse=True)
```

```
for ms in corpus_tries:  
    print(ms["cote"], ms["date"])  
# MS. Ashmole 1511 1210  
# MS 24 1200
```

sorted() renvoie une **nouvelle** liste : **corpus** n'est pas modifié.



Commenter son code

Améliorer la lisibilité

- Le code dit **ce que** fait le programme. Le commentaire explique **pourquoi** il le fait ainsi.
- Il éclaire les choix qui ne sont pas évidents : un seuil, une exception, une convention propre au corpus.

Faciliter la maintenance et la collaboration

- On comprend plus vite un code commenté quand il faut le corriger ou le mettre à jour.
- Les autres membres d'un projet peuvent comprendre votre logique et y contribuer.

Documenter pour soi-même et pour les autres

- **Mémoire personnelle** : dans trois mois, vous aurez oublié pourquoi vous avez fait ce choix.
- **Transmission** : le code d'un projet de recherche est souvent repris par d'autres chercheurs.

Exemple : expliquer un choix

```
elif date <= 1200:  
    # Un siècle va de l'an 1 à l'an 100 :  
    # l'an 1200 appartient donc au XIIe siècle, pas au XIIIe.  
    resultat = "XIIe siècle"
```

Le code seul ne dit pas pourquoi on a écrit `<=` plutôt que `<`. Le commentaire le dit.





Les commentaires avec

Syntaxe

- Tout ce qui suit # sur une ligne est **ignoré** par Python.
- Un commentaire peut occuper **sa propre ligne**, au-dessus du code qu'il explique, ou se placer **en fin de ligne**, après deux espaces.

```
# Datation approximative : le catalogue indique « vers 1210 »  
manuscrit_oxford["date"] = 1210  
manuscrit_oxford["numerise"] = True # vérifié sur le site de la  
Bodleian
```

Bonnes pratiques

- **Pertinence** : un commentaire ajoute une information, il ne paraphrase pas le code.
- **Actualisation** : quand le code change, le commentaire change aussi. Un commentaire faux est pire que pas de commentaire.
- **Clarté** : des phrases simples et directes.

```
creatures.append("phénix") # ajoute phénix à la liste ←  
inutile
```

Trois usages courants

Signaler une tâche à faire ou un problème connu

TODO : vérifier la datation d'Oxford dans le catalogue.
FIXME : decrire_creatures() écrit « 1 créatures » au singulier.

TODO signale ce qui reste à faire. **FIXME** signale un défaut connu, à corriger. VS Code repère ces mots-clés.

Expliquer une logique non évidente

Un siècle va de l'an 1 à l'an 100 :
l'an 1200 appartient donc au XIIe siècle.

Désactiver temporairement du code

print(manuscrit) # affichage de contrôle, désactivé
Dans VS Code, **Ctrl + /** (ou **Cmd + /** sur Mac) commente ou décommente les lignes sélectionnées.



Les docstrings

Définition

Une **docstring** est une chaîne de caractères placée **juste après** la ligne `def`, entre triples guillemets `"""..."""`. Ce n'est pas un commentaire : Python la **conserve** et peut l'afficher. On en écrit pour :

- les **fonctions** : ce qu'elles font, leurs paramètres, ce qu'elles renvoient ;
- les **modules** : en tête du fichier `.py`, une vue d'ensemble de son contenu (slide 23) ;
- les **classes**, qu'on verra en séance 3.

Conventions (PEP 257)

La PEP 257 est le texte officiel qui fixe les conventions des docstrings.

- Utiliser des triples guillemets **doubles** `"""`.
- Écrire une **première ligne** de résumé courte, à l'impératif.
- Si l'on développe, **sauter une ligne** après le résumé.

Lire une docstring

`help(siecle)`

affiche la signature `siecle(date)` puis la docstring entière. Dans VS Code, elle s'affiche aussi au **survol** du nom de la fonction. `help()` fonctionne aussi avec les fonctions de Python : essayez `help(len)` ou `help(str.split)`.

Raccourci VS Code

`Shift + Option + A` sur Mac, ou `Shift + Alt + A` sur Windows et Linux, entoure la sélection de triples guillemets `"""..."""`. C'est pratique pour écrire une docstring.



Notions clés à retenir

Les fonctions

- **Définir** : `def nom(parametres):`, suivi d'un bloc indenté. Une fonction réalise une tâche et se réutilise à volonté.
- **Fonction ou méthode** : `len(titre)` s'oppose à `titre.split()`. La méthode appartient à un type de donnée.
- **Renvoyer** : `return` renvoie une valeur réutilisable, `print()` ne fait que l'afficher. Sans `return`, une fonction renvoie `None`.
- **Paramètres** : arguments positionnels ou nommés, valeurs par défaut. On ne crée un paramètre que pour une information qui change d'un appel à l'autre.

Les fonctions (suite)

- **Nombre variable d'arguments** : `*args` regroupe les arguments dans un tuple, `**kwargs` dans un dictionnaire.
- **Portée** : une variable créée dans une fonction est locale et disparaît après l'appel.
- **lambda** : une petite fonction anonyme en une ligne.

Documenter son code

- `#` pour expliquer **pourquoi**, ainsi que `TODO` et `FIXME`.
- Les docstrings `"""..."""` pour décrire fonctions et modules, lisibles avec `help()`.

Hackathon AN x pictoria



16-18 décembre : Archives nationales de Pierrefitte-sur-Seine

Détails et inscriptions

MINISTÈRE DE LA CULTURE | ARCHIVES NATIONALES x pictoria | MSH MONDES

16-18 DÉCEMBRE 2026
PIERREFITTE-SUR-SEINE - SAINT-DENIS

Métro 13 station Saint-Denis Université
59, rue Guymer
Pierrefitte-sur-Seine
93383 Saint-Denis

HACKATHON

ARCHIVES NATIONALES × PICTORIA

EXPLORATION MULTIMODALE DES ARCHIVES
ICONOGRAPHIQUES ET AUDIOVISUELLES

NOUVEAUX REGARDS

ARCHIVES + INTELLIGENCE ARTIFICIELLE + CO-CONSTRUCTION

Scannez pour en savoir plus

pictoria
CONSORTIUM HUMA-NUM

PROGRAMME D'IDENTIFICATION, CLASSIFICATION, TRAITEMENT D'IMAGES, OBSERVATION ET RECONNAISSANCE DES FORMES PAR L'INTELLIGENCE ARTIFICIELLE